Handling Too-Hard Contexts in Automatic Curriculum Learning for Contextual Reinforcement Learning

Maximilian Schwingel
Laurenz von Schilgen
Connor Gröling
github.com/m111xi/RL_Team_Hamelner

1 Motivation

Automatic curriculum learning is often motivated by the idea that training should adapt to the current ability of the agent rather than follow a fixed manual schedule. In contextual reinforcement learning, this becomes especially relevant when some contexts are learnable early while others are temporarily too difficult, which makes naive uniform sampling potentially inefficient. The central question of this project is therefore not only how to detect difficult contexts, but how a curriculum should react once such contexts are identified.

2 Related Topics

This project is related to contextual MDPs, generalization in reinforcement learning, and automatic curriculum learning. It also connects to PPO as a standard policy optimization method and to benchmark settings such as CARL, where environment variations are represented through explicit contexts. More broadly, the project is situated in the literature on adaptive sampling, prioritized replay, and curriculum design for reinforcement learning.

3 Idea

The main idea is to study how automatic context curricula should handle contexts that are currently too hard during training. Instead of comparing several broad curriculum definitions at once, we fix a single difficulty signal and focus on the decision that follows from it: what should the curriculum do when a context crosses a hardness threshold?

We therefore want to compare different handling policies for hard contexts, such as temporarily excluding them, down-weighting them, or revisiting them later with increased priority. The goal is to isolate this decision dimension and to understand which reaction leads to the best trade-off between sample efficiency, final performance, and training stability.

$$\pi_\theta \in \Pi, \qquad \pi_\theta : \mathcal{S} \times \mathcal{C} \mapsto \mathcal{A} \tag{1}$$

4 Experimental Setup

Difficulty Signal We will use one fixed difficulty signal throughout the project, most likely an error-based signal such as moving average loss, prediction error, or failure rate on a context. This choice keeps the project small and interpretable, since the goal is not to develop a new difficulty estimator but to evaluate what should happen after difficulty has been detected.

Base Sampling Scheme The base sampler will be uniform context sampling. This serves as a transparent baseline and avoids baking an easy-to-hard progression directly into the experiment. All handling policies will therefore be evaluated on top of the same sampling backbone.

Handling Policies Whenever a context is classified as too hard according to the fixed difficulty signal, one of the following policies will be applied:

- **Ignore:** keep sampling uniformly and do not react to the hard-context signal.
- **Hard exclude:** temporarily pause contexts classified as too hard until their score improves.
- **Down-weight + revisit:** reduce the sampling probability of hard contexts, but continue revisiting them occasionally.
- **Replay-style prioritization:** keep hard contexts in the sampling pool and adapt their probability according to their estimated learning potential.

Environments & Metrics We will use one CARL environment as the main testbed, with a second environment only if time permits for a robustness check. Suitable candidates are contextual CartPole as a simpler environment and contextual LunarLander as a more challenging one. We plan to measure episodic return, sample efficiency, robustness across seeds, and recovery of hard contexts over time.

Comparison Protocol All methods will share the same agent, optimizer, training budget, seeds, and difficulty signal. The only manipulated factor will be the handling policy for hard contexts. This way, the project isolates the effect of hard-case handling instead of comparing complete curriculum frameworks.

5 Analysis

Besides the main performance comparison, we also want to include a sanity-check analysis of the difficulty signal itself. In particular, we will examine whether the chosen signal correlates with empirical task difficulty and whether this relationship helps explain the differences between the handling policies. This should provide a more mechanistic interpretation of the results rather than only reporting final scores.

6 Expected Contribution

The contribution of the project is not a completely new curriculum algorithm. Instead, it is a focused empirical study of a concrete design choice inside automatic curriculum learning: how to handle contexts that are currently too difficult. This makes the project appropriate for a small course project, while still yielding interpretable insights into curriculum behavior.

7 Timeline

We expect the project to proceed in the following stages:

- Literature review and implementation planning: 3–4 days
- Core implementation: 1–1.5 weeks
- Initial experiments and debugging: 1 week
- Final runs and analysis: 4–5 days
- Slides and final presentation: 3–4 days